

HW/Lab 2 Sol: SNR, BER, Sampling, Hardware

ECSE 4560/6560: Modern Communication Systems

RPI Fall 2025

Matlab Simulations: BPSK BER Simulation in AWGN

```
1 %% BPSK BER Simulation
2 %
3 % This script simulates the BER of coherent BPSK over an AWGN channel
4 % and compares it to the analytical expression  $Q(\sqrt{2*Eb/N0}) = 0.5*erfc(\sqrt{Eb/N0})$ .
5 %
6 % COMPLETE ALL TODOs.
7 % For theory, follow https://piazza.com/class\_profile/get\_resource/mevvk9kirhg2fu/mfwtorbg5i25kj
8 %
9 % Q1: Simulate BER vs SNR for coherent BPSK and compare to analytical
10 % curve.
11 % Q2: (6560 only) Repeat with "oversampling" factors L in {1,2,4} by
12 % reducing per-sample
13 % noise variance by 1/L (processing-gain modeling).
14 % IMPORTANT MODELING NOTE:
15 % — We use the common simulation normalization  $E_b = 1$  (dimensionless)
16 % so that
17 %  $N_0 = E_b / (E_b/N_0)$  is also treated numerically as a dimensionless
18 % quantity.
19 % — In "Part 2", we model an "oversampling factor" (oversamp_fac) by
20 % reducing the
21 % *effective* noise variance per sample by 1/oversamp_fac. This
22 % mimics the idea
23 % that more samples per bit allow lower per-sample noise if total
24 % noise energy
25 % over a bit is held fixed. Practically, this behaves like a
26 % processing gain
```

```

20 % and shows a BER improvement as oversamp_fac increases.
21 %


---


22
23 clear; clc;
24
25 %% ----- Simulation Parameters


---


26 num_bits = 1e5; % Number of BPSK symbols (bits) to transmit in
    each trial
27 snr_db_vec = 0:1:10; % Vector of Eb/N0 points in dB to simulate
28 num_snrs = numel(snr_db_vec);
29
30 % Generate random BPSK symbols in {-1, +1}
31 % bits = 0/1 -> map to -1/+1 via 2*bits - 1
32 bits = randi([0 1], 1, num_bits);
33 bpsk_syms = 2*bits - 1;
34
35 %% ----- Part 1: Single "oversampling" setting


---


36 % We simulate BER for a single oversampling factor (oversamp_fac = 1).
37 % With oversamp_fac = 1, the "effective" noise variance equals the
    baseline.
38 oversamp_fac = 1; % No oversampling-based noise scaling
39 ber_vec = zeros(1, num_snrs); % Preallocate simulated BER array
40
41 for i = 1:num_snrs
42     % Select Eb/N0 for this run
43     snr_db = snr_db_vec(i); % Eb/N0 (dB)
44     ebn0 = 10^(snr_db/10); % Eb/N0 (linear)
45
46     % Energy per bit (normalized): Eb = 1 (dimensionless)
47     % Under this normalization, N0 = Eb / (Eb/N0) is numerically
        dimensionless too.
48     eb = 1;
49     n0 = eb / ebn0; % "PSD-like" number under the
        normalization
50
51     % Effective noise scaling due to oversampling:
52     % We reduce the per-sample noise variance by oversamp_fac (
        processing gain model).
53     % For *real* BPSK baseband, the per-sample *variance* from "N0" is
        N0/2.
54     % Here we scale that variance by 1/oversamp_fac.
55     n0_effective = n0 / oversamp_fac;
56
57     % Generate real AWGN samples: variance = (n0_effective)/2

```

```

58     % randn() has variance 1, so scale by sqrt(variance)
59     n_vec = sqrt(n0_effective/2) * randn(1, num_bits);
60
61     % Pass through AWGN channel
62     noisy_syms = bpsk_syms + n_vec;
63
64     % Hard-decision detection for BPSK:
65     % Decision rule: y >= 0 -> +1, else -> -1
66     rx_syms = ones(1, num_bits);           % initialize to +1
67     rx_syms(noisy_syms < 0) = -1;          % flip to -1 where negative
68
69     % Compute BER:
70     % An error occurs where transmitted and detected symbols differ.
71     % |rx - tx| is 0 for correct decisions, 2 for errors (since values
72       are +-1).
73     % So, number_of_errors = sum(|rx - tx|/2).
74     ber_vec(i) = sum(abs(rx_syms - bpsk_syms)/2) / num_bits;
75 end
76
77 % Analytical BER for coherent BPSK in AWGN:
78 % Pb = Q( sqrt(2*Eb/N0) ) = 0.5*erfc( sqrt(Eb/N0) ).
79 snr_lin = 10.^(snr_db_vec/10);
80 ber_vec_analytical = 0.5 * erfc( sqrt(snr_lin) );
81
82 % Plot (Part 1): Simulated vs Analytical
83 figure(1); clf;
84 semilogy(snr_db_vec, ber_vec, 'LineWidth', 2, 'LineStyle', ':'); hold
85 on;
86 semilogy(snr_db_vec, ber_vec_analytical, 'LineWidth', 2);
87 legend('Simulated BER', 'Analytical Expression', 'Location', 'southwest
88 ');
89 title('BER Curves for BPSK (Normalized Eb=1)');
90 xlabel('Eb/N0 (dB)');
91 ylabel('BER');
92 grid on; grid minor;
93
94 %% ----- Part 2: "Oversampling" sweep via effective-noise scaling
95
96 % Here we sweep oversamp_fac in {1, 2, 4} by *reducing* the per-sample
97   noise
98 % variance linearly with oversamp_fac. This models a processing-gain-
99   like effect.
100 % As oversamp_fac increases, you will see a BER improvement (by design)
101 .
102
103 % Oversampling-like scaling values
104 oversamp_fac_vec = [1, 2, 4];
105 ber_mat = zeros(numel(oversamp_fac_vec), num_snrs); % BER for each
106   oversamp + SNR

```

```

98
99 for osamp_idx = 1:numel(oversamp_fac_vec)
100     oversamp_fac = oversamp_fac_vec(osamp_idx);
101
102     for i = 1:num_snrs
103         % Eb/N0 selection
104         snr_db = snr_db_vec(i);
105         ebn0    = 10^(snr_db/10);
106
107         % Normalized Eb and corresponding "N0"
108         eb = 1;
109         n0 = eb / ebn0;
110
111         % Apply the oversampling noise scaling (processing-gain model)
112         n0_effective = n0 / oversamp_fac;
113
114         % Generate real AWGN with variance = (n0_effective)/2
115         n_vec = sqrt(n0_effective/2) * randn(1, num_bits);
116
117         % Transmit over AWGN channel and detect
118         noisy_syms = bpsk_syms + n_vec;
119
120         rx_syms = ones(1, num_bits);
121         rx_syms(noisy_syms < 0) = -1;
122
123         % BER for this (oversamp_fac, SNR) pair
124         ber_mat(osamp_idx, i) = sum(abs(rx_syms - bpsk_syms)/2) /
            num_bits;
125     end
126 end
127
128 % Plot (Part 2): Effect of "oversampling" (via noise scaling) on BER
129 figure(2); clf;
130 semilogy(snr_db_vec, ber_mat(1,:), 'LineWidth', 2, 'LineStyle', ':');
131     hold on;
132 semilogy(snr_db_vec, ber_mat(2,:), 'LineWidth', 2, 'LineStyle', ':');
133 semilogy(snr_db_vec, ber_mat(3,:), 'LineWidth', 2, 'LineStyle', ':');
134 legend('Simulated BER, Oversamp 1', ...
135         'Simulated BER, Oversamp 2', ...
136         'Simulated BER, Oversamp 4', ...
137         'Location', 'southwest');
138 xlabel('Eb/N0 (dB)');
139 ylabel('BER');
140 title('Modeled Effect of "Oversampling" on BPSK BER (Noise-Scaling View
    )');
141 grid on; grid minor;

```

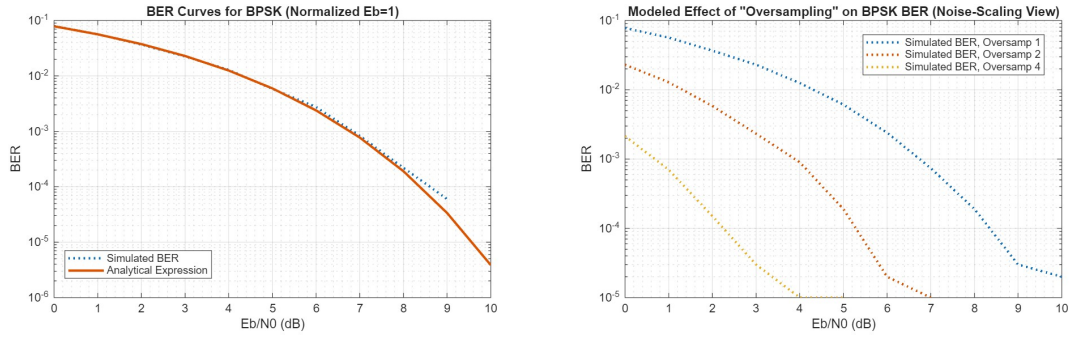


Figure 1: (a) We can observe BER decreases as SNR increases (b) Oversampling reduces BER

Hardware Experiment

1. Simulink Model

```

1 % Parameters
2 fs = 1e6; % sample rate
3 sps = 10; % samples per symbol
4 fc = 915e6; % carrier frequency (inside unlicensed band)
5 n = 1e5; % number of bits or symbols per frame (1 bit per symbol)
6 m = 5; % Simulink will run (m+1) frames
7 G = 65; % input power gain in dB
8 A = -70; % transmitter attenuation in dB
9 D = 2; % input delay
10
11 % Load model
12 md = 'lab2.slx';
13 load_system(md);
14
15 % run simulation
16 out = sim(md, 'StopTime', num2str(n*sps/fs*5));
17 % only-noise frame
18 Pn = mean(out.s_r.Data{3}.^2); % noise power
19 % last frame
20 Ps = (mean(out.s_r.Data{end}.^2)-Pn)/2; % divided by 2 because
    empirically q=0.5
21 th = mean(out.s_r.Data{end})/2; % decision threshold
22
23 snr = pow2db(Ps/Pn); % SNR
24 ber = mean(out.s_r.Data{end}<th); % BER

```

2. Plot SNR vs BER

```

1 % Parameters
2 fs = 1e6; % sample rate
3 sps = 10; % samples per symbol

```

```

4  fc = 915e6; % carrier frequency (inside unlicensed band)
5  n = 1e5; % number of bits or symbols per frame (1 bit per symbol)
6  m = 5; % Simulink will run (m+1) frames
7  A = -70; % transmitter attenuation in dB
8  D = 2; % input delay
9  G_set = 0:0.5:10; % input power gain
10
11 % Initialization
12 BER = zeros(21,1);
13 SNR = zeros(21,1);
14
15 % Load model
16 md = 'lab2.slx';
17 load_system(md);
18
19 % run simulation
20 for i=1:21
21     G = G_set(i);
22     % run simulation
23     out = sim(md, 'StopTime', num2str(n*sps/fs*5));
24     % only-noise frame
25     Pn = mean(out.s_r.Data{3}.^2); % noise power
26     % last frame
27     Ps = (mean(out.s_r.Data{end}.^2)-Pn)/2; % divided by 2 because
        empirically q=0.5
28     th = mean(out.s_r.Data{end})/2; % decision threshold
29
30     SNR(i) = pow2db(Ps/Pn); % SNR
31     BER(i) = mean(out.s_r.Data{end}<th); % BER
32     disp(i)
33 end
34 %%
35 plot(SNR,BER, 'o')
36 xlabel('SNR(dB)')
37 ylabel('BER')
38 title('SNR(dB) vs BER under OOK')

```